vit

Typst slides with View Transitions

a tutorial — press →

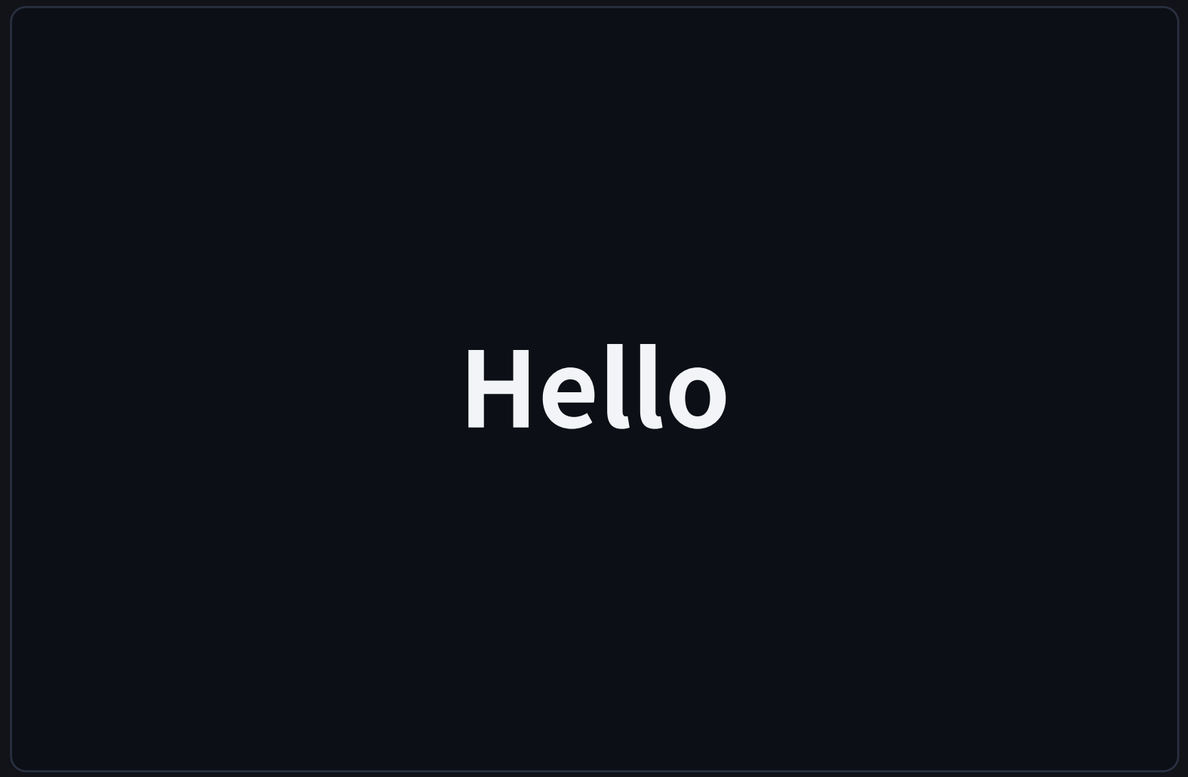
A deck in ten lines

`deck` is the document, `slide` is a page, `mark` names what should travel. Everything else is ordinary Typst.

```
#import "@preview/vit:0.1.0": *

#deck(title: "My talk")[
  #slide[
    #set align(center + horizon)
    #mark("hi")[#text(size: 52pt)[Hello]]
  ]
  #slide[
    #mark("hi")[#text(size: 24pt)[Hello]]
    — and the rest of the page arrives around it.
  ]
]
```

```
typst compile --root . talk.typ talk.pdf
typst compile --root . --features html talk.typ talk.html
```



One source, two compiles: the PDF is the handout, the HTML is the deck, pixel for pixel.

A deck in ten lines

`deck` is the document, `slide` is a page, `mark` names what should travel. Everything else is ordinary Typst.

```
#import "@preview/vit:0.1.0": *

#deck(title: "My talk")[
  #slide[
    #set align(center + horizon)
    #mark("hi")[#text(size: 52pt)[Hello]]
  ]
  #slide[
    #mark("hi")[#text(size: 24pt)[Hello]]
    — and the rest of the page arrives around it.
  ]
]
```

```
typst compile --root . talk.typ talk.pdf
typst compile --root . --features html talk.typ talk.html
```

Hello

— and the rest of the page arrives around it. Two ordinary Typst pages; the only addition is the name on the word that should travel.

One source, two compiles: the PDF is the handout, the HTML is the deck, pixel for pixel.

One question, three answers

Did the layout change, or is one object moving? The answer picks the API.

Transition

The layout changed: another page, or the next frame of this one.

```
slide(..frames)
mark("key")[…]
deck/slide/mark(transition:)
```

View Transitions: the browser pairs marks by key.

One question, three answers

Did the layout change, or is one object moving? The answer picks the API.

Transition

The layout changed: another page, or the next frame of this one.

```
slide(..frames)
mark("key")[…]
deck/slide/mark(transition:)
```

View Transitions: the browser pairs marks by key.

Element animation

One object varies with a parameter: the wave at its next phase.

```
mark("key", s0, s1, …)
```

Web Animations, on the SVG nodes; stepped with → and ←.

One question, three answers

Did the layout change, or is one object moving? The answer picks the API.

Transition

The layout changed: another page, or the next frame of this one.

```
slide(..frames)
mark("key")[…]
deck/slide/mark(transition:)
```

View Transitions: the browser pairs marks by key.

Element animation

One object varies with a parameter: the wave at its next phase.

```
mark("key", s0, s1, …)
```

Web Animations, on the SVG nodes; stepped with → and ←.

Continuous animation

Something moving on its own, with nobody pressing anything.

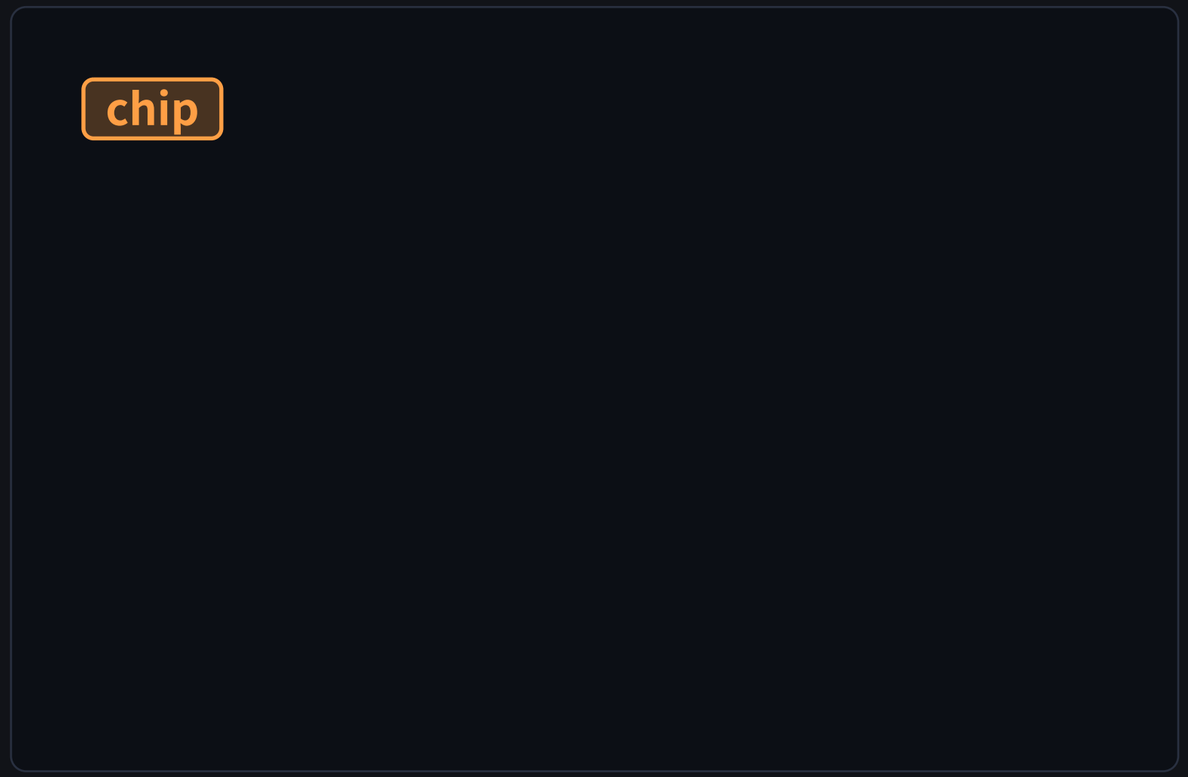
```
tween(play: …) · waapi.animate
```

Web Animations, running while the page rests here.

mark: one name, one object

The same key on two adjacent frames or pages is **one object**. The browser pairs the boxes by name and interpolates position, size and colour.

```
#slide(
  title: "Chips",
  [#place(dx: 20pt, dy: 20pt,
     mark("chip")[#chip(amber, 22pt)])],
  [#place(dx: 170pt, dy: 130pt,
     mark("chip")[#chip(gold, 34pt)])],
  [#place(dx: 330pt, dy: 250pt,
     mark("chip")[#chip(cyan, 26pt)])],
)
```

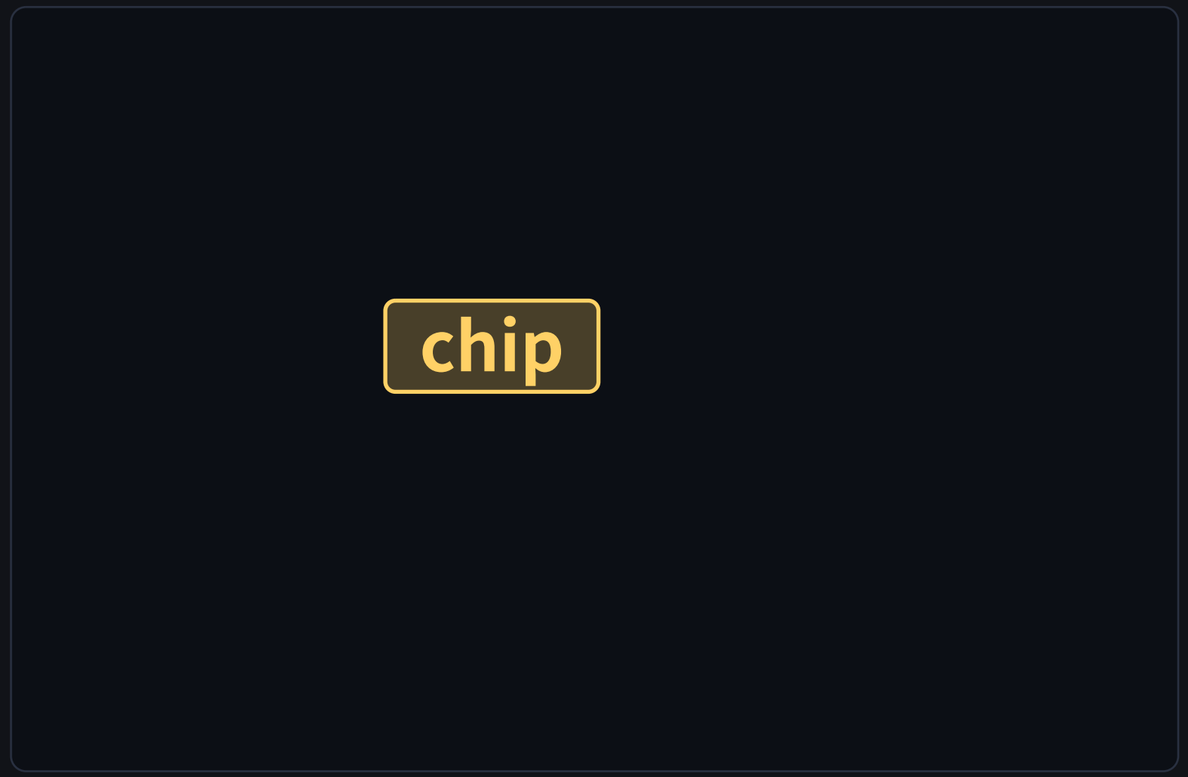


Identity is the key and only the key — nothing is inferred from the content.

mark: one name, one object

The same key on two adjacent frames or pages is **one object**. The browser pairs the boxes by name and interpolates position, size and colour.

```
#slide(
  title: "Chips",
  [#place(dx: 20pt, dy: 20pt,
     mark("chip")[#chip(amber, 22pt)])],
  [#place(dx: 170pt, dy: 130pt,
     mark("chip")[#chip(gold, 34pt)])],
  [#place(dx: 330pt, dy: 250pt,
     mark("chip")[#chip(cyan, 26pt)])],
)
```

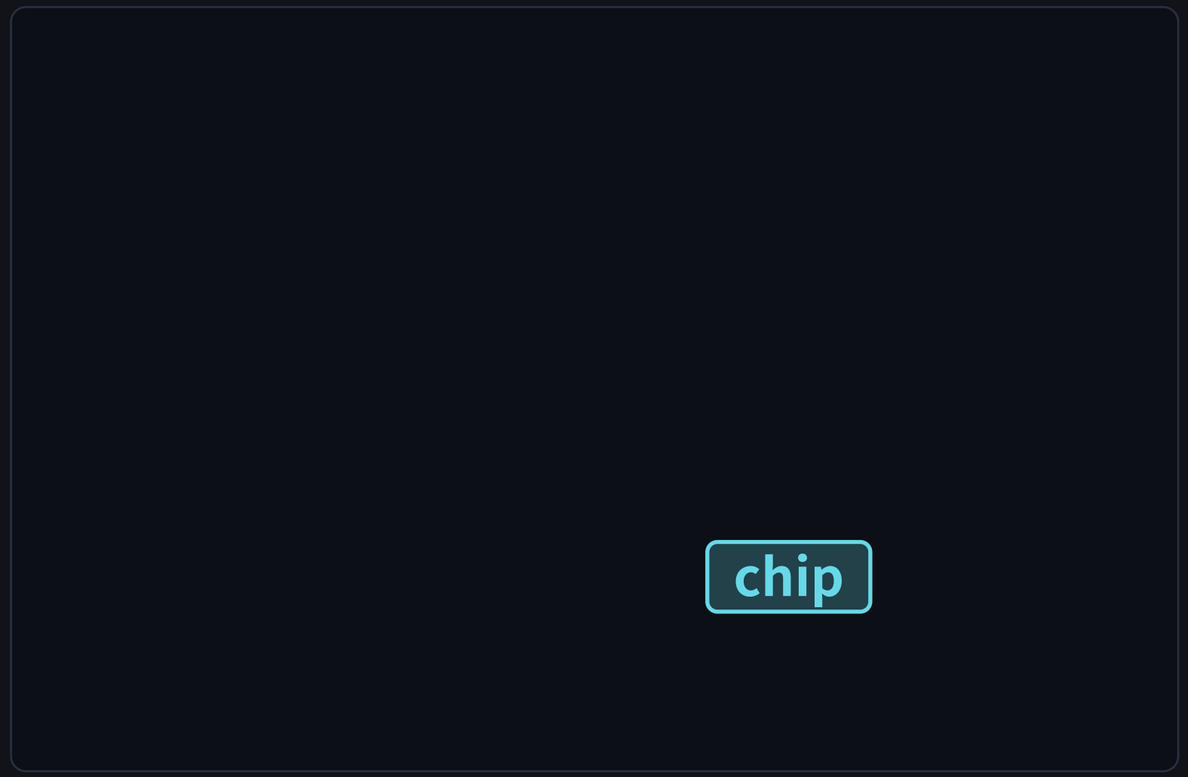


Identity is the key and only the key — nothing is inferred from the content.

mark: one name, one object

The same key on two adjacent frames or pages is **one object**. The browser pairs the boxes by name and interpolates position, size and colour.

```
#slide(
  title: "Chips",
  [#place(dx: 20pt, dy: 20pt,
     mark("chip")[#chip(amber, 22pt)])],
  [#place(dx: 170pt, dy: 130pt,
     mark("chip")[#chip(gold, 34pt)])],
  [#place(dx: 330pt, dy: 250pt,
     mark("chip")[#chip(cyan, 26pt)])],
)
```



Identity is the key and only the key — nothing is inferred from the content.

Frames and positions

Several bodies in one `slide` are **frames of one page**: one thumbnail, one dot each, `#5.1` in the address bar, and one PDF page each.

```
#let a = [ … ]
#slide(
  title: "Frames",     // the thumbnail's caption
  note: [what to say], // the desk and the speaker view
  a,
  a + b,
  a + b + c,
)
```

Frames are for “one more line”. The page cross-fades and the marks morph: never a page jump.

Frames and positions

Several bodies in one `slide` are **frames of one page**: one thumbnail, one dot each, `#5.2` in the address bar, and one PDF page each.

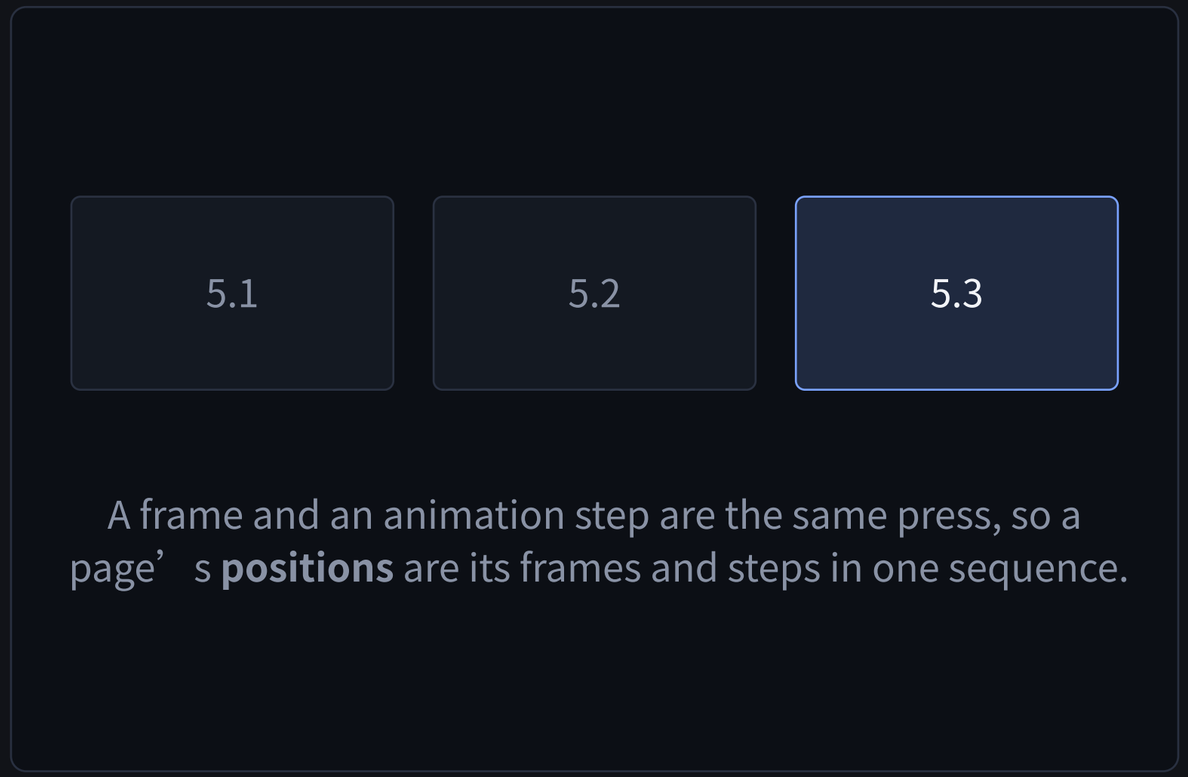
```
#let a = [ … ]
#slide(
  title: "Frames",     // the thumbnail's caption
  note: [what to say], // the desk and the speaker view
  a,
  a + b,
  a + b + c,
)
```

Frames are for “one more line”. The page cross-fades and the marks morph: never a page jump.

Frames and positions

Several bodies in one `slide` are **frames of one page**: one thumbnail, one dot each, `#5.3` in the address bar, and one PDF page each.

```
#let a = [ … ]
#slide(
  title: "Frames",     // the thumbnail's caption
  note: [what to say], // the desk and the speaker view
  a,
  a + b,
  a + b + c,
)
```



Frames are for “one more line”. The page cross-fades and the marks morph: never a page jump.

build: frames that accumulate

`build(a, b, c)` is the three frames `a`, `a + b`, `a + b + c`. Content in, content out — it builds the body of a drawing just as well.

```
#slide(title: "How it works", ..build(
  item("measure", blue, "every marked box…"),
  item("hoist", amber, "each one into an SVG host…"),
  item("pair", green, "old against new, by name…"),
))
```

> **measure** — every marked box, before anything moves

`build` when the later parts are **meant** to push the layout. When they must not, the next page.

build: frames that accumulate

`build(a, b, c)` is the three frames `a`, `a + b`, `a + b + c`. Content in, content out — it builds the body of a drawing just as well.

```
#slide(title: "How it works", ..build(
  item("measure", blue, "every marked box…"),
  item("hoist", amber, "each one into an SVG host…"),
  item("pair", green, "old against new, by name…"),
))
```

measure — every marked box, before anything moves

hoist — each one into an SVG host with a name

`build` when the later parts are **meant** to push the layout. When they must not, the next page.

build: frames that accumulate

`build(a, b, c)` is the three frames `a`, `a + b`, `a + b + c`. Content in, content out — it builds the body of a drawing just as well.

```
#slide(title: "How it works", ..build(
  item("measure", blue, "every marked box…"),
  item("hoist", amber, "each one into an SVG host…"),
  item("pair", green, "old against new, by name…"),
))
```

measure — every marked box, before anything moves

hoist — each one into an SVG host with a name

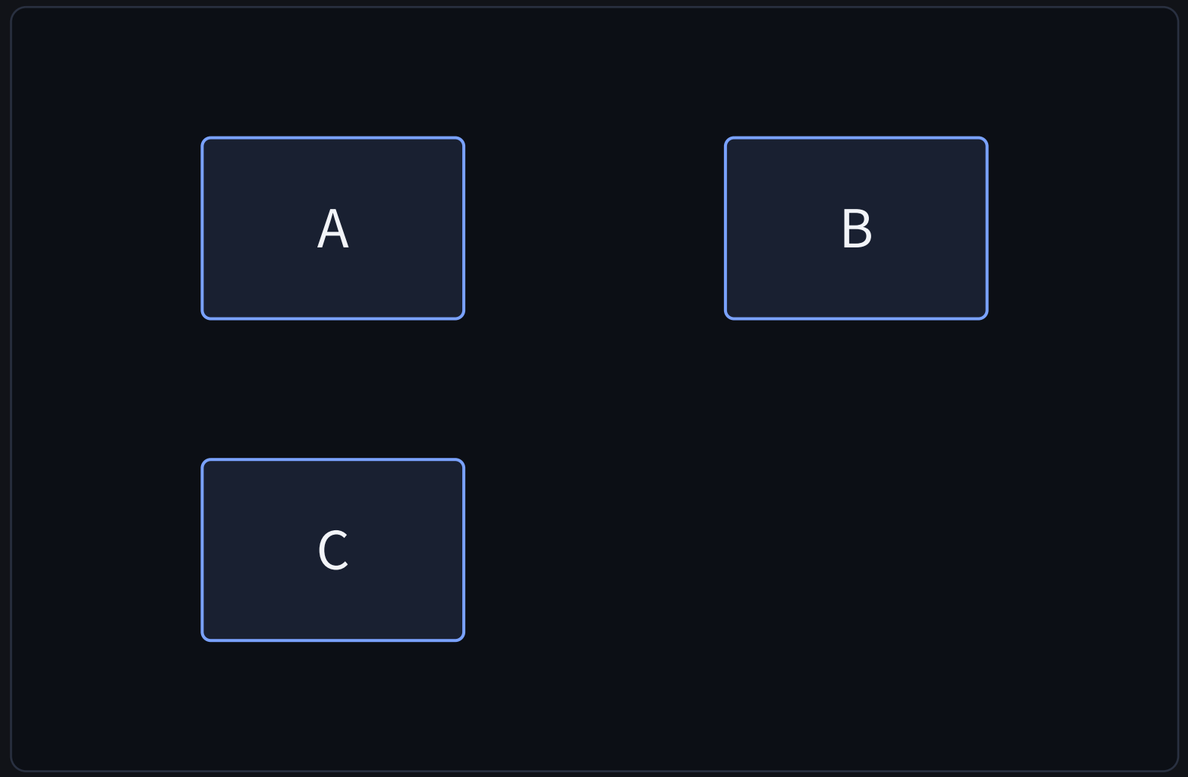
pair — old against new, by name — the browser's own job

`build` when the later parts are **meant** to push the layout. When they must not, the next page.

reveal: frames that stay put

`reveal(n, (step, at) => …)` writes the frames once. `at(k, x)` is x from frame k on; before that it **keeps x's space**, so the layout never moves.

```
#slide(..reveal(3, (step, at) => box({
  place(dx: 0pt,   dy: 0pt,   cell[A])
  place(dx: 260pt, dy: 0pt,   cell[B])
  place(dx: 0pt,   dy: 160pt, cell[C])
  place(dx: 260pt, dy: 160pt,
    at(2, mark("d", transition: "zoom", cell[D])))
  place(dx: 138pt, dy: 44pt,       // holds its box
    at(2, line(length: 114pt)))
  rect(.., stroke: at(3, 1pt))     // else none
})))
```

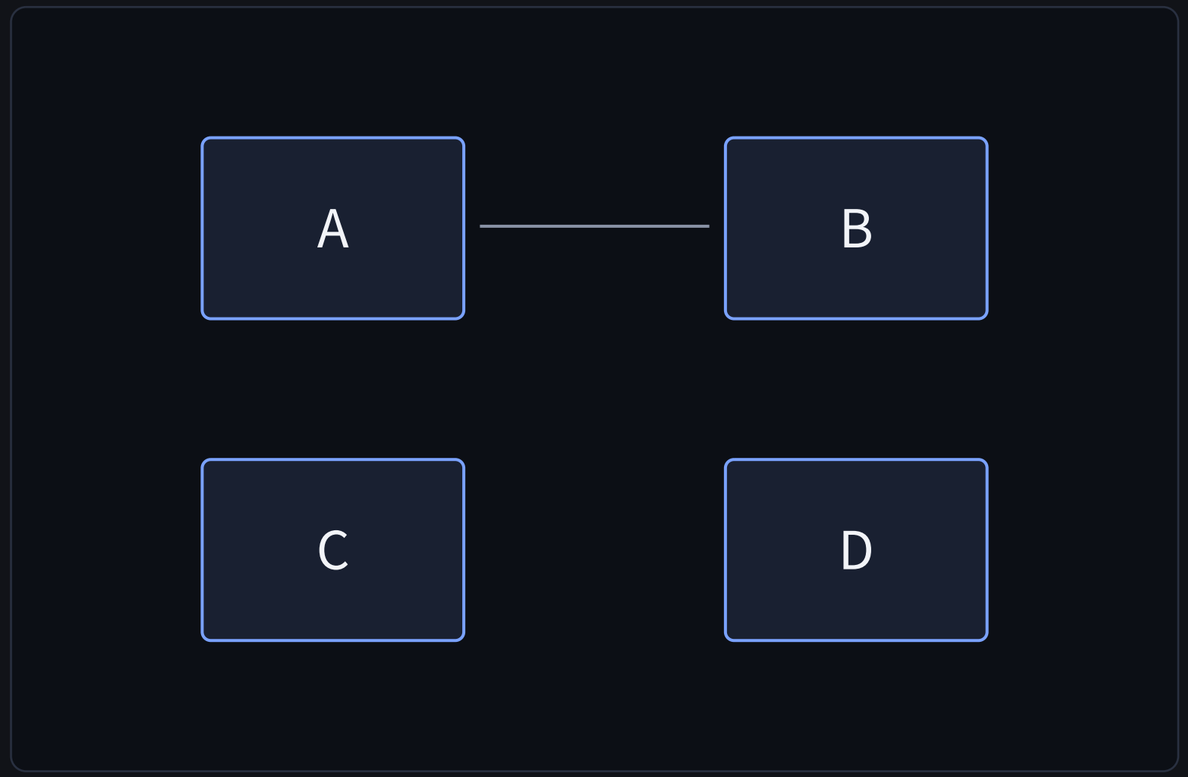


What has not arrived is `hiden` and holds its box; anything else is `none` — a stroke switched off.

reveal: frames that stay put

`reveal(n, (step, at) => …)` writes the frames once. `at(k, x)` is x from frame k on; before that it **keeps x's space**, so the layout never moves.

```
#slide(..reveal(3, (step, at) => box({
  place(dx: 0pt,   dy: 0pt,   cell[A])
  place(dx: 260pt, dy: 0pt,   cell[B])
  place(dx: 0pt,   dy: 160pt, cell[C])
  place(dx: 260pt, dy: 160pt,
    at(2, mark("d", transition: "zoom", cell[D])))
  place(dx: 138pt, dy: 44pt,       // holds its box
    at(2, line(length: 114pt)))
  rect(.., stroke: at(3, 1pt))     // else none
})))
```

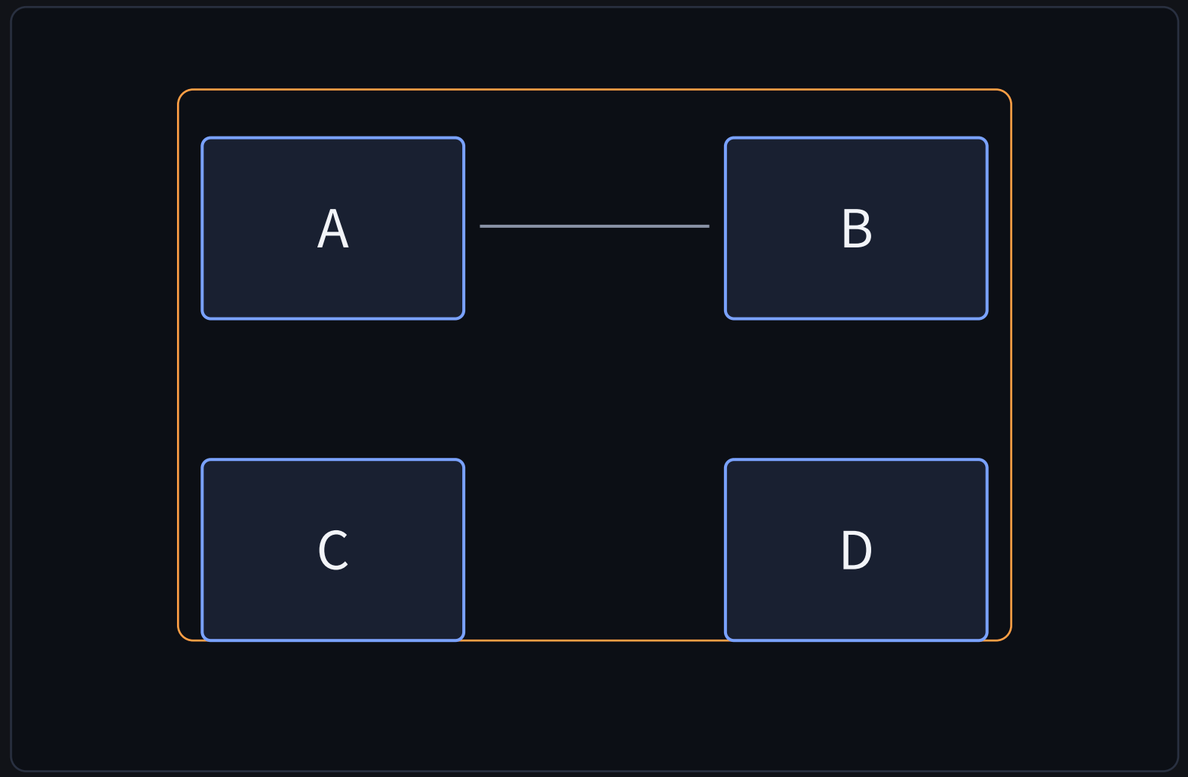


What has not arrived is `hide`n and holds its box; anything else is `none` — a stroke switched off.

reveal: frames that stay put

`reveal(n, (step, at) => …)` writes the frames once. `at(k, x)` is x from frame k on; before that it **keeps x's space**, so the layout never moves.

```
#slide(..reveal(3, (step, at) => box({
  place(dx: 0pt,   dy: 0pt,   cell[A])
  place(dx: 260pt, dy: 0pt,   cell[B])
  place(dx: 0pt,   dy: 160pt, cell[C])
  place(dx: 260pt, dy: 160pt,
    at(2, mark("d", transition: "zoom", cell[D])))
  place(dx: 138pt, dy: 44pt,       // holds its box
    at(2, line(length: 114pt)))
  rect(.., stroke: at(3, 1pt))     // else none
})))
```



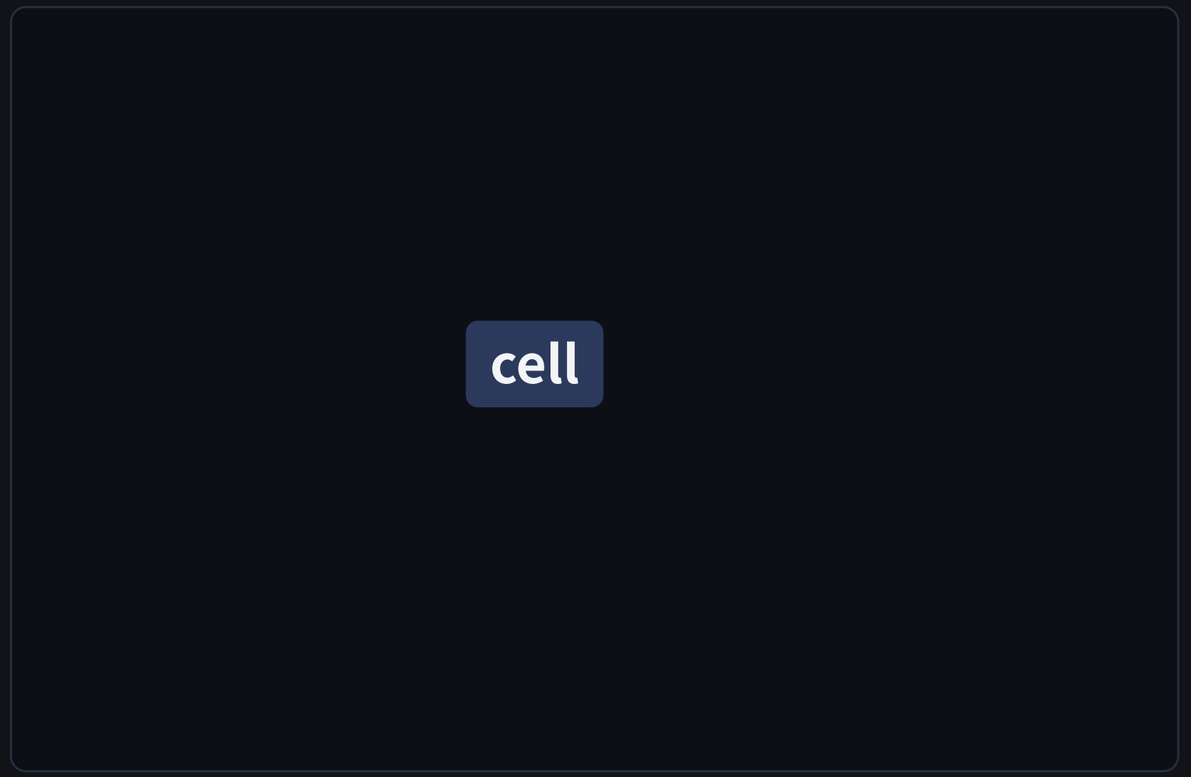
What has not arrived is `hide`n and holds its box; anything else is `none` — a stroke switched off.

Split and merge

A key that stands once on this frame and three times on the next **splits** into three; three to one **merges**. The counts need not divide.

```
#let at(x, y) = place(dx: x, dy: y, mark("cell")[#cell])

#slide(
  at(210pt, 140pt),                      // one
  at(10pt, 20pt) + at(390pt, 250pt) + at(200pt, 130pt),
  at(60pt, 250pt) + at(350pt, 30pt),     // three to two
  at(210pt, 140pt),                      // back to one
)
```



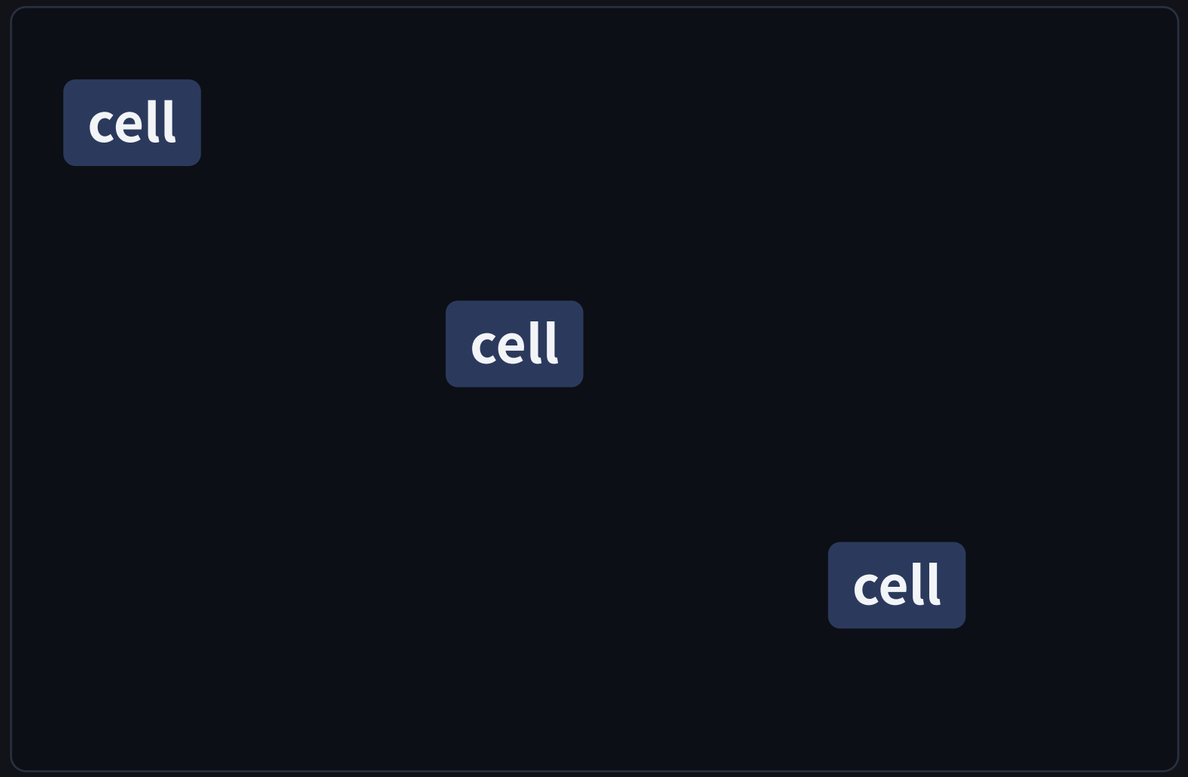
Names carry an occurrence index (`m-cell-1`, `m-cell-2`), so one key may stand several times.

Split and merge

A key that stands once on this frame and three times on the next **splits** into three; three to one **merges**. The counts need not divide.

```
#let at(x, y) = place(dx: x, dy: y, mark("cell")[#cell])

#slide(
  at(210pt, 140pt),                      // one
  at(10pt, 20pt) + at(390pt, 250pt) + at(200pt, 130pt),
  at(60pt, 250pt) + at(350pt, 30pt),     // three to two
  at(210pt, 140pt),                      // back to one
)
```



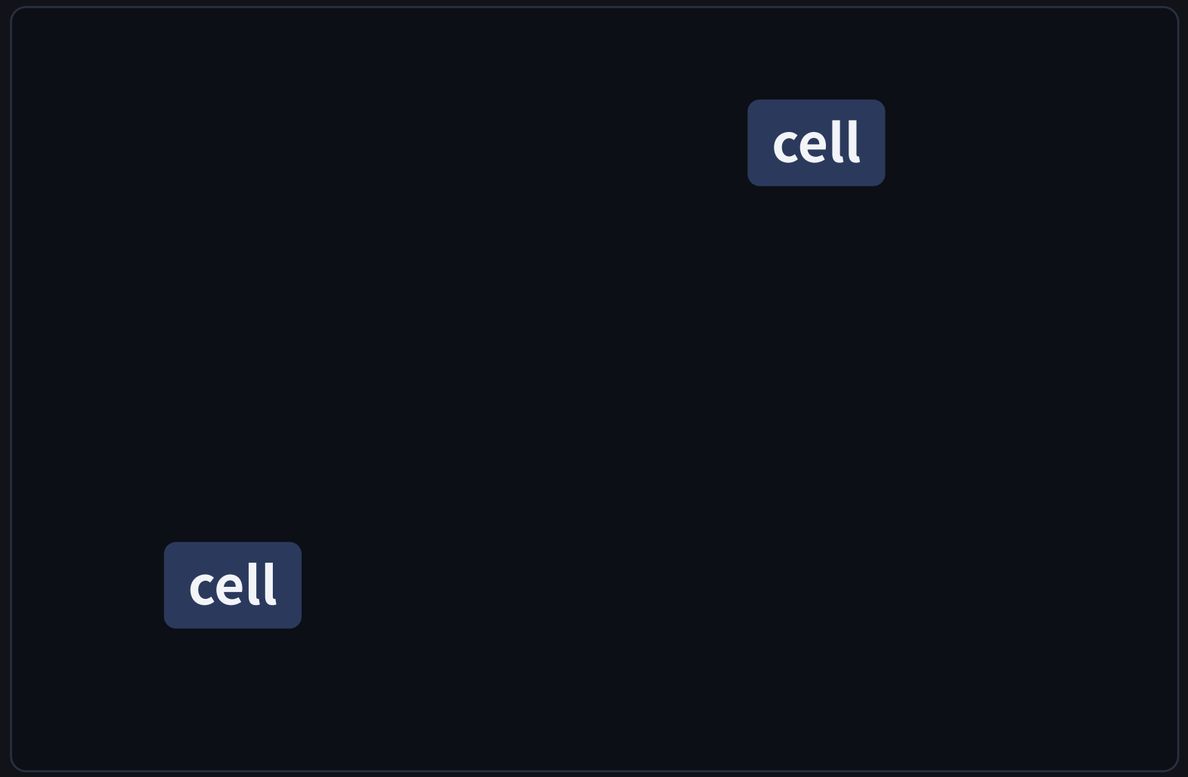
Names carry an occurrence index (`m-cell-1`, `m-cell-2`), so one key may stand several times.

Split and merge

A key that stands once on this frame and three times on the next **splits** into three; three to one **merges**. The counts need not divide.

```
#let at(x, y) = place(dx: x, dy: y, mark("cell")[#cell])

#slide(
  at(210pt, 140pt),                      // one
  at(10pt, 20pt) + at(390pt, 250pt) + at(200pt, 130pt),
  at(60pt, 250pt) + at(350pt, 30pt),     // three to two
  at(210pt, 140pt),                      // back to one
)
```



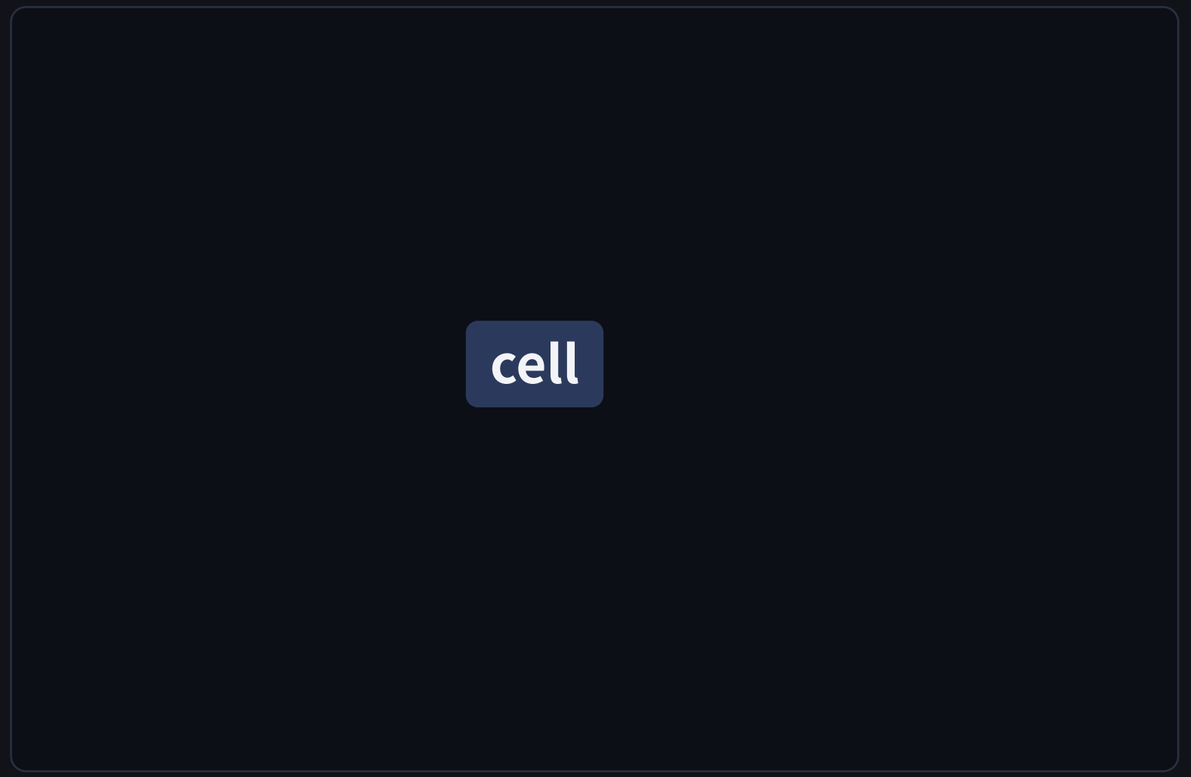
Names carry an occurrence index (`m-cell-1`, `m-cell-2`), so one key may stand several times.

Split and merge

A key that stands once on this frame and three times on the next **splits** into three; three to one **merges**. The counts need not divide.

```
#let at(x, y) = place(dx: x, dy: y, mark("cell")[#cell])

#slide(
  at(210pt, 140pt),                      // one
  at(10pt, 20pt) + at(390pt, 250pt) + at(200pt, 130pt),
  at(60pt, 250pt) + at(350pt, 30pt),     // three to two
  at(210pt, 140pt),                      // back to one
)
```

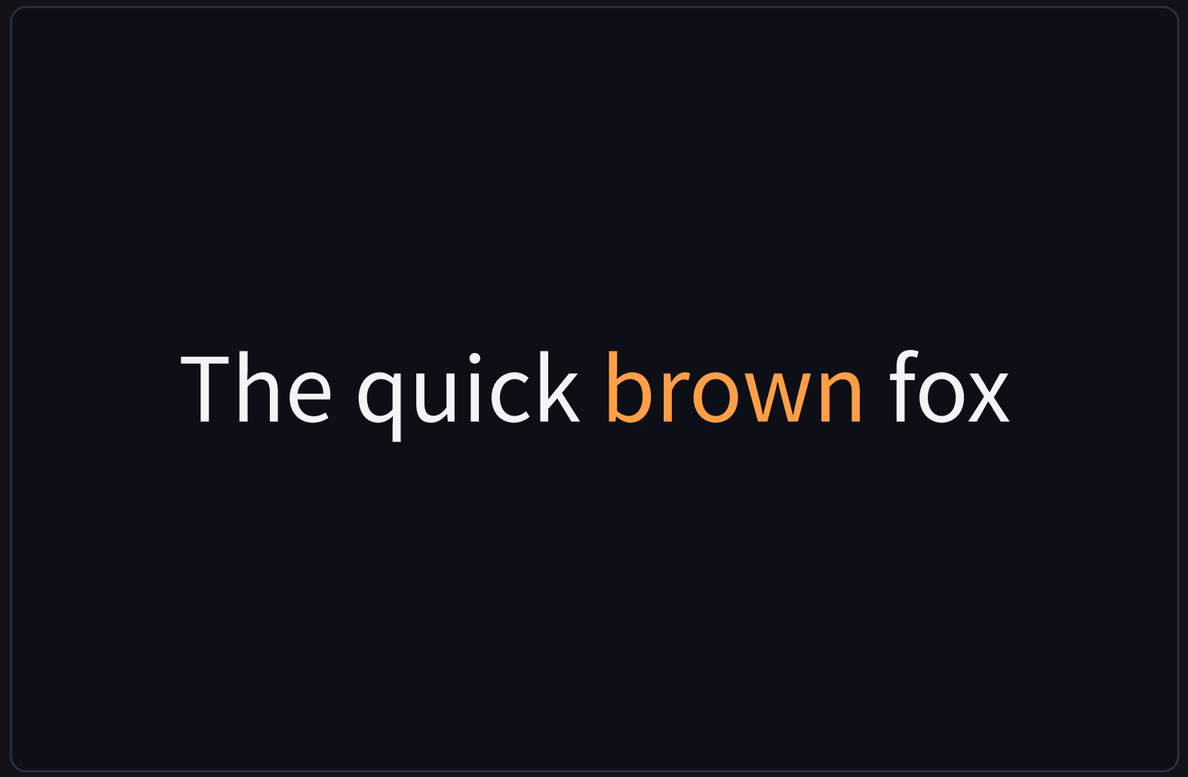


Names carry an occurrence index (`m-cell-1`, `m-cell-2`), so one key may stand several times.

Marks nest

A mark inside a mark is a group of its own. The outer one carries what is left once the inner ones are lifted out.

```
#let nest(sz, al) = [
  #set align(al)
  #mark("outer")[#text(size: sz)[
    The quick #mark("inner")[#text(fill: amber)[brown]] fox
  ]]
]
#slide(title: "Nested marks",
  nest(44pt, center + horizon),
  nest(24pt, left + top),
)
```



Nesting is how one change’ s result takes part in the next **as a whole** — see `layers`.

Marks nest

A mark inside a mark is a group of its own. The outer one carries what is left once the inner ones are lifted out.

```
#let nest(sz, al) = [
  #set align(al)
  #mark("outer")[#text(size: sz)[
    The quick #mark("inner")[#text(fill: amber)[brown]] fox
  ]]
]
#slide(title: "Nested marks",
  nest(44pt, center + horizon),
  nest(24pt, left + top),
)
```

Nesting is how one change’ s result takes part in the next **as a whole** — see `layers`.

How a thing enters

`mark(transition:)` is **this object's own** entrance and exit. It is used only where the mark is one-sided; a mark that is on both sides morphs regardless.

```
#let definition = mark("def", transition: "wipe-down",
  env(blue)[#definition(number: "1")[…]])
#let cauchy = mark("thm", transition: "wipe-up",
  env(amber)[#theorem(number: "2")[…]])
#let cauchy-proof = mark("proof",
  transition: (enter: "wipe-left", leave: "zoom"),
  env(green)[#proof[…]])

#slide(..build(definition, cauchy, cauchy-proof))
```

> **Definition 1 (Cauchy sequence).** A sequence (a_n) is **Cauchy** if for every $\varepsilon > 0$ there is an N with $|a_m - a_n| < \varepsilon$ whenever $m, n > N$.

One object, one effect: any occurrence may give it, and two may not disagree.

How a thing enters

`mark(transition:)` is **this object's own** entrance and exit. It is used only where the mark is one-sided; a mark that is on both sides morphs regardless.

```
#let definition = mark("def", transition: "wipe-down",
  env(blue)[#definition(number: "1")[…]])
#let cauchy = mark("thm", transition: "wipe-up",
  env(amber)[#theorem(number: "2")[…]])
#let cauchy-proof = mark("proof",
  transition: (enter: "wipe-left", leave: "zoom"),
  env(green)[#proof[…]])

#slide(..build(definition, cauchy, cauchy-proof))
```

Definition 1 (Cauchy sequence). A sequence (a_n) is **Cauchy** if for every $\varepsilon > 0$ there is an N with $|a_m - a_n| < \varepsilon$ whenever $m, n > N$.

Theorem 2. A real sequence converges if and only if it is Cauchy.

One object, one effect: any occurrence may give it, and two may not disagree.

How a thing enters

`mark(transition:)` is **this object's own** entrance and exit. It is used only where the mark is one-sided; a mark that is on both sides morphs regardless.

```
#let definition = mark("def", transition: "wipe-down",
  env(blue)[#definition(number: "1")[…]])
#let cauchy = mark("thm", transition: "wipe-up",
  env(amber)[#theorem(number: "2")[…]])
#let cauchy-proof = mark("proof",
  transition: (enter: "wipe-left", leave: "zoom"),
  env(green)[#proof[…]])

#slide(..build(definition, cauchy, cauchy-proof))
```

Definition 1 (Cauchy sequence). A sequence (a_n) is **Cauchy** if for every $\varepsilon > 0$ there is an N with $|a_m - a_n| < \varepsilon$ whenever $m, n > N$.

Theorem 2. A real sequence converges if and only if it is Cauchy.

Proof. A Cauchy sequence is bounded, so it has a convergent subsequence, and the Cauchy condition pulls the whole sequence to that limit. □

One object, one effect: any occurrence may give it, and two may not disagree.

Page transitions

`deck(transition:)` is the default for turning to another page; `slide(transition:)` overrides it for one page. A string is the same effect both ways, `(enter:, leave:)` names the sides.

```
#deck(transition: "fade")[…]     // the default
#slide(transition: "slide")[…]   // this page only
#slide(transition: (enter: "wipe-up", leave: "fade"))[…]
#deck(transition: none)[…]       // no page transitions
```

`fade`	cross-fade — the default
`slide`	horizontal push; forward, the new page comes from the right
`rise`	vertical push, in from the bottom
`zoom`	the new one shrinks into place from three times life size, out of focus, and lands sharp
`wipe-left` `wipe-right` `wipe-up` `wipe-down`	reveal, named by the direction the front travels
`none`	that side switches at once, while the transition still runs

Page effects govern **unmarked** content, and only between pages. `transition: none` turns them off.

Settings ride along

The same dictionary that names the effects carries the settings for **this** transition. Settings beside the effects belong to both sides; settings inside a side belong to that side and beat them.

```
#slide(transition: (effect: "zoom", duration: 900, zoom: 4))[…]

#slide(transition: (
  enter: (effect: "slide", duration: 200, push: -100%),
  leave: (effect: "fade", duration: 900),
  easing: (0.4, 0, 0.2, 1),
))[…]

#mark("word", transition: (effect: "rise", duration: 400))[…]
```

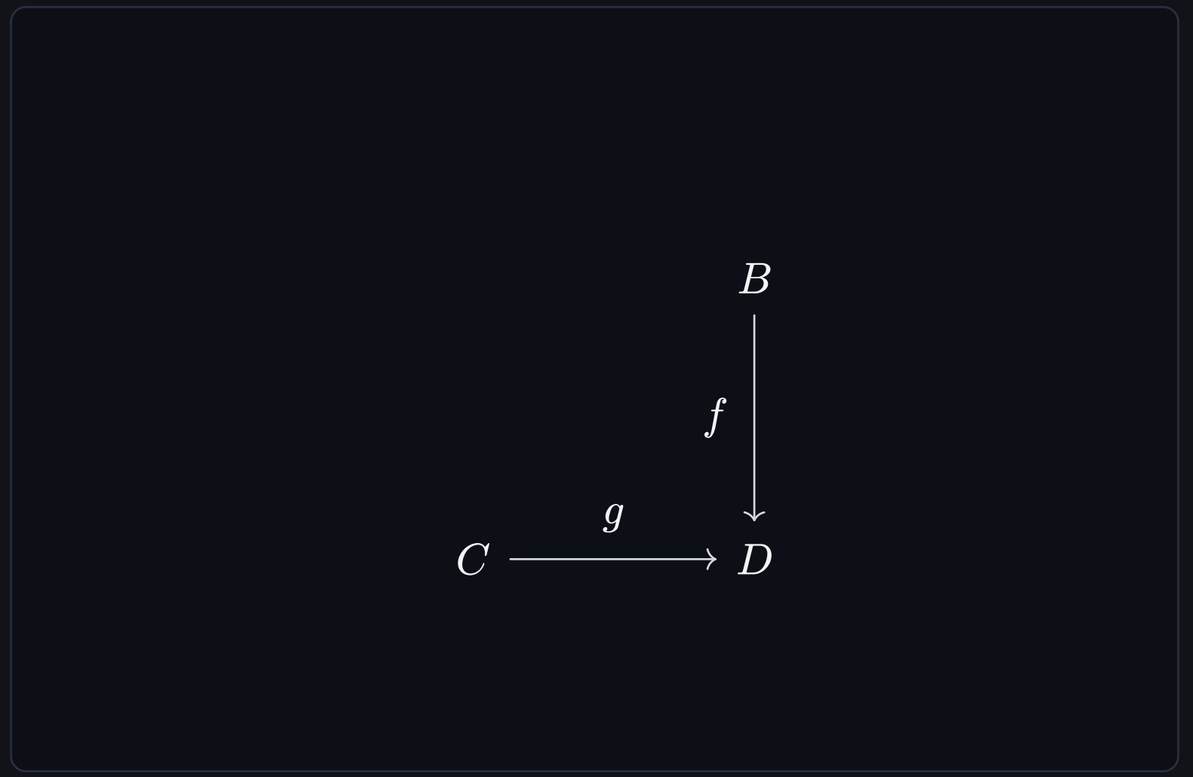
`duration`	milliseconds, as in deck(duration:)
`easing`	the four numbers of a cubic Bézier
`zoom`	how many times life size the zoom starts at
`push`	how far slide and rise travel; negative goes the other way
`fit`	"none" draws both images at their own size instead of stretching them into the box
`anchor`	the corner they are pinned to when fit is none

A setting is a variable `deck.css` reads, so a typo is a compile error. Deck, slide and mark all take the same dictionary.

Assemblies: layers

A group interpolates its **box** and cross-fades its two **pictures** — what is inside them is never moved. So what moves has to be one picture.

```
#layers("pb", meet: bottom + right,
  cd({                      // the square, written once
    node((1, 0), $B$); node((1, 1), $D$)
    node((0, 0), at(2, mark("pbA", transition: "zoom")[$A$]))
    edge((0, 0), (1, 0), at(2, $p$), "->",
         stroke: at(2, 1pt + grey))
  }),
  if step >= 3 { cd({       // X, a layer of its own,
    node((0, 0), hide($A$))    // anchors drawn hidden
    node((-1, -1), mark("pbX", transition: "rise")[$X$])
  }) },
)
```

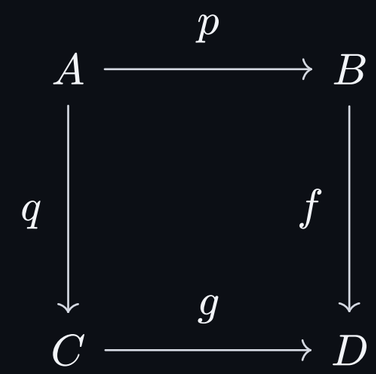


`reveal` keeps a picture from being re-laid-out; `layers` is for what arrives **outside** it.

Assemblies: layers

A group interpolates its **box** and cross-fades its two **pictures** — what is inside them is never moved. So what moves has to be one picture.

```
#layers("pb", meet: bottom + right,
  cd({                      // the square, written once
    node((1, 0), $B$); node((1, 1), $D$)
    node((0, 0), at(2, mark("pbA", transition: "zoom")[$A$]))
    edge((0, 0), (1, 0), at(2, $p$), "->",
         stroke: at(2, 1pt + grey))
  }),
  if step >= 3 { cd({       // X, a layer of its own,
    node((0, 0), hide($A$))    // anchors drawn hidden
    node((-1, -1), mark("pbX", transition: "rise")[$X$])
  }) },
)
```

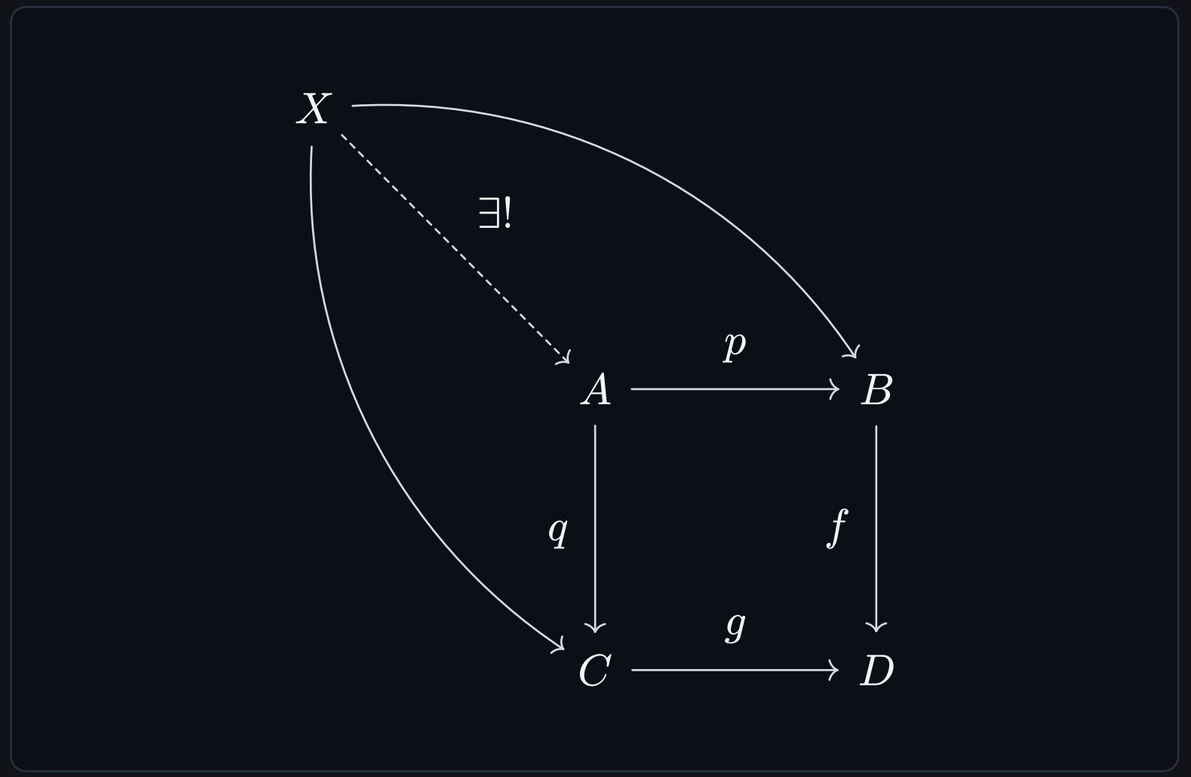


`reveal` keeps a picture from being re-laid-out; `layers` is for what arrives **outside** it.

Assemblies: layers

A group interpolates its **box** and cross-fades its two **pictures** — what is inside them is never moved. So what moves has to be one picture.

```
#layers("pb", meet: bottom + right,
  cd({                       // the square, written once
    node((1, 0), $B$); node((1, 1), $D$)
    node((0, 0), at(2, mark("pbA", transition: "zoom")[$A$]))
    edge((0, 0), (1, 0), at(2, $p$), "->",
         stroke: at(2, 1pt + grey))
  }),
  if step >= 3 { cd({      // X, a layer of its own,
    node((0, 0), hide($A$))    // anchors drawn hidden
    node((-1, -1), mark("pbX", transition: "rise")[$X$])
  }) },
)
```



`reveal` keeps a picture from being re-laid-out; `layers` is for what arrives **outside** it.

Element animation

Several bodies are the states of one drawing: `tween(s0, s1, …)`, or `states(…)` inside a canvas, where only what moves is a state. A drawing needs no `mark` — a mark is identity between pages, and this is motion inside one. → moves to the next, ← back along the same path, and the browser interpolates the SVG nodes.

```
#import "@preview/tween:0.1.0": compat
#let cz = compat.cetz.tweened(cetz)   // once, per file

#let wave = cetz.canvas({
  import cetz.draw: circle, line, rect, content
  let (states,) = cz
  rect((-0.5, -2.1), (8.5, 2.1), stroke: none)  // pin the box
  line((0, 0), (8, 0))              // the axis never moves
  states(..range(0, 6).map(t => {   // …these do
    let amp = 0.3 + 0.18 * t
    let f(x) = amp * calc.sin(1.2 * x - 0.5 * t)
    line(..range(0, 41).map(i => (i * .2, f(i * .2))))
    circle((1.4 * t, f(1.4 * t)), radius: .25, fill: …)
    content((4, -1.7), [t = #t])
  }))
})

#slide[#wave]
```

The states are **one drawing under different parameters**: write `f(t)`, feed it a few values.

What interpolates

The states' SVG nodes are compared one by one, and every property that differs is handed to `el.animate()`. Paths that are not the same list of commands are reconciled first.

```
// each is one canvas, and inside it one states(…)
states(..range(3, 9).map(k => polygon((0, 0), k)))  // a side more
states(..range(0, 6).map(k => line(..walk.slice(0, k + 2))))
states(..range(0, 3).map(k =>                        // a line
  if k == 0 { line((0, 0), (4, 0)) }                 // becomes
  else { bezier((0, 0), (4, 0), c1(k), c2(k)) }))    // a curve
```

`none` against a colour becomes `transparent`. What has no in-between — text, an arc — cross-fades.

Designing the states

Two paths interpolate point for point, so **you** choose where the new vertices come from: draw every state with the same points and stack the spare ones where they should start.

```
#let counts = (6, 12, 24, 48, 96)
#let N = counts.last()      // one budget for all states
#let corners(k, r) = range(N).map(j => {
  let n = counts.at(k)      // j-th point of the n-gon,
  let a = 2 * calc.pi * calc.quo(j * n, N) / n
  (r * calc.cos(a), r * calc.sin(a))   // spares stacked
})
// edges drawn as cubics: a zero-length *line* is dropped
#states(..range(0, 5).map(exhaust))   // inside the canvas
```

Same for the labels: all five counts are drawn in every state, π keeps four decimals.

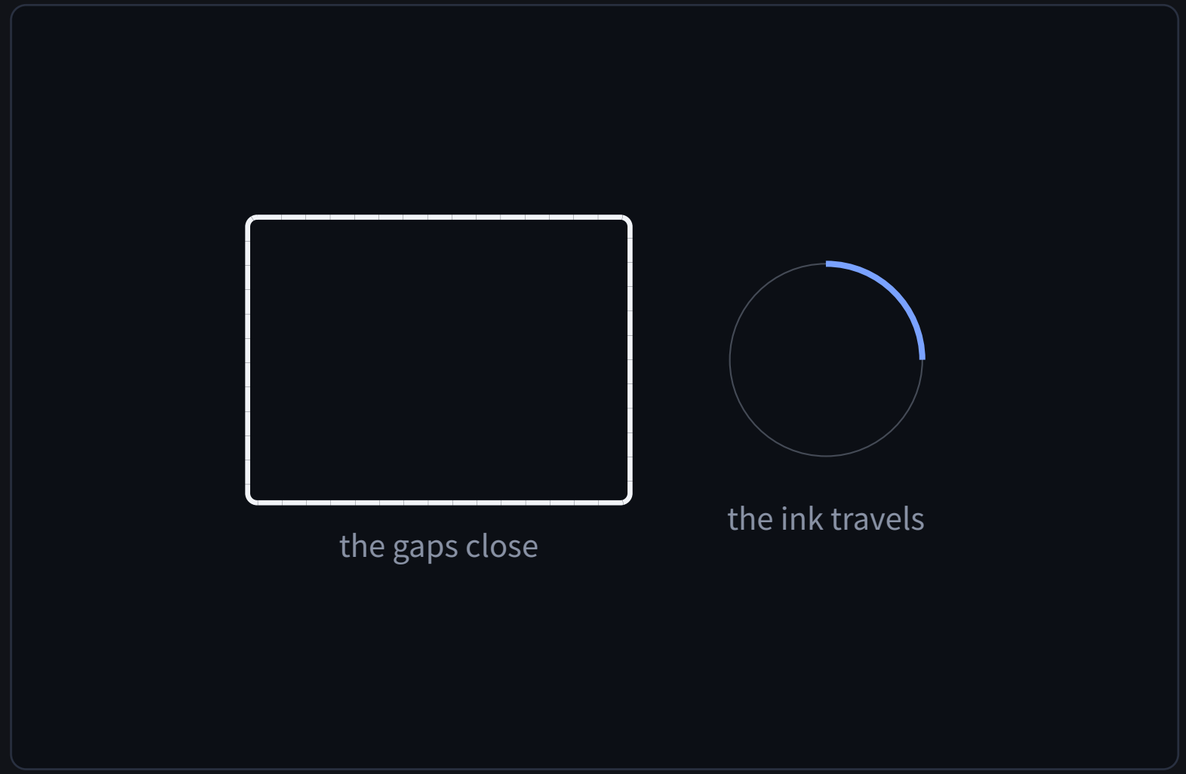
Dashes

`stroke-dasharray` interpolates like any other property. It is worth reaching for when what you want is **not** expressible as points: a pattern, or ink that moves along a path that stays still.

```
// gaps close: the period stays 12pt, so the dashes grow
// forward into their own gaps instead of sliding along
#let frame(f) = rect(..., stroke: (dash: (
  array: (6pt + 6pt * f, 6pt - 6pt * f), phase: 0pt)))

// ink placed anywhere on the path: an empty dash, a gap
// up to where it starts, the ink, a gap longer than the
// path. Never `phase:` — Typst writes it to
// stroke-dashoffset without flipping the sign, and the
// PDF and the browser then disagree.
#let ring = cetz.canvas(length: 1cm, {
  let (circle, over) = cz
  circle((0, 0), radius: R, stroke: (dash: (
    array: over(..range(5).map(j => (0pt,
      j / 4 * (CIRC - LEN) * 1cm, LEN * 1cm,
      2 * CIRC * 1cm))), phase: 0pt)))
})

#tween(..range(5).map(j => frame(j / 4)))
#ring
```

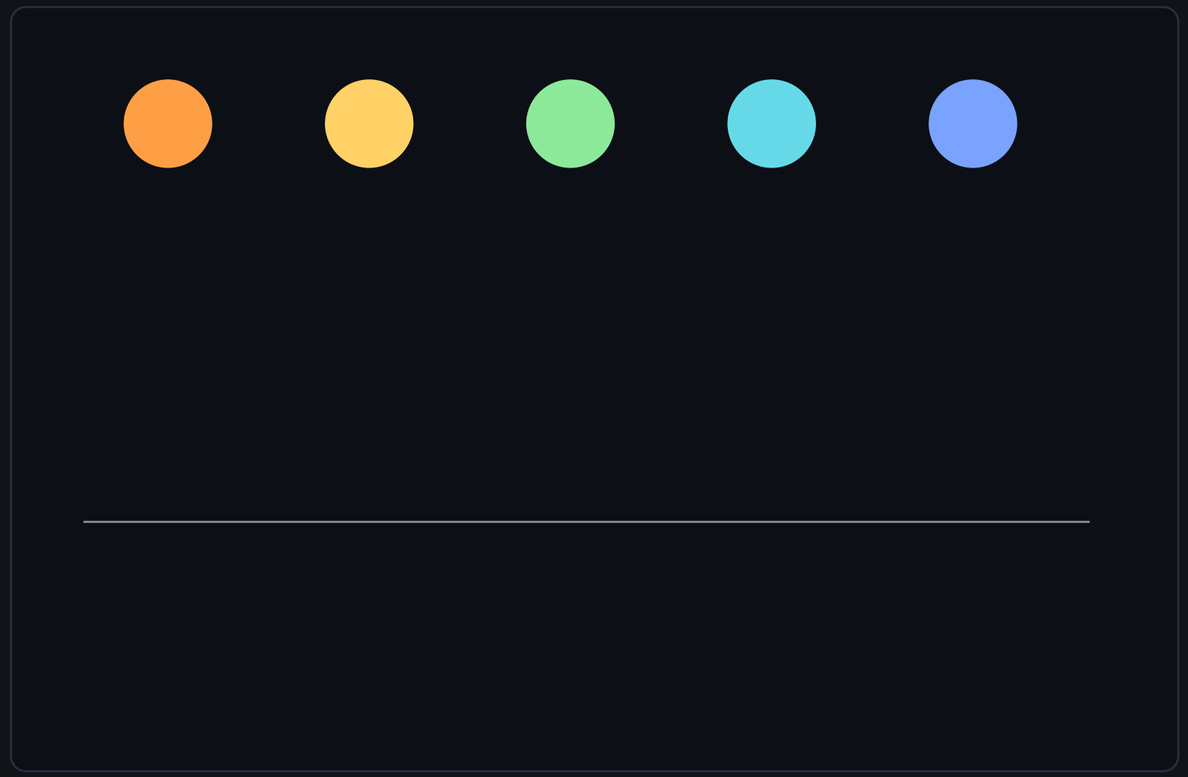


Not for a line that simply grows from its end: give the states their points and let `paths.js` pad the shorter one. Reach for a dash when points cannot say it. Every state must carry the attribute — write the solid box as a zero gap, not as no dash — or the states differ in their attributes and the whole node cross-fades instead.

Continuous animation

`waapi.animate` is Web Animations from Typst: keyframes and options as the API takes them, handed to `el.animate()` as they are. No DSL of our own, and no deck required — it moves the same figure in a blog post.

```
#let ball(i) = (
  keyframes: (
    (transform: "translateY(0)",
     easing: "cubic-bezier(.45, 0, 1, .55)"),
    (transform: "translateY(420%) scale(1.3, .7)",
     offset: .5, easing: "linear"),
    (transform: "translateY(0)"),
  ),
  duration: 1500, delay: i * 140,
)
#import "@preview/tween:0.1.0": waapi
#for i in range(5) {
  place(…, waapi.animate(..ball(i))[…])
}
```

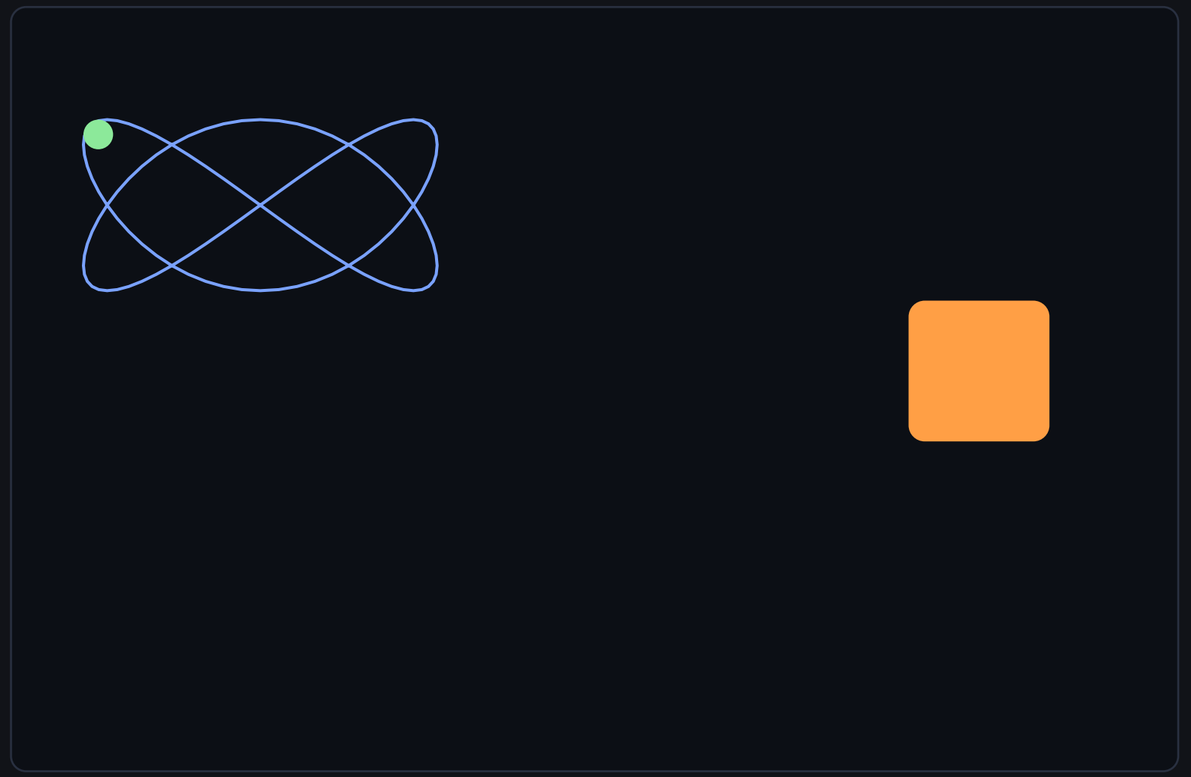


`duration`, `delay`, `easing`, `direction`, `iterations`; nothing plays under `prefers-reduced-motion`.

follow: an element along a path

`waapi.track` names a path and `follow:` runs an element's centre along it, on the compositor. It is the one convenience in the binding.

```
#place(dx: 20pt, dy: 30pt, waapi.track("orbit", cetz.canvas({
  import cetz.draw: line
  line(..lissajous, close: true)
})))
#place(dx: 20pt, dy: 30pt, waapi.animate(
  follow: "orbit", duration: 4000,
)[#std.circle(radius: .3cm)])
#place(dx: 430pt, dy: 120pt, waapi.animate(
  keyframes: ((transform: "rotate(0)"),
              (transform: "rotate(1turn)")),
  duration: 6000,
)[#std.rect(…)])
```



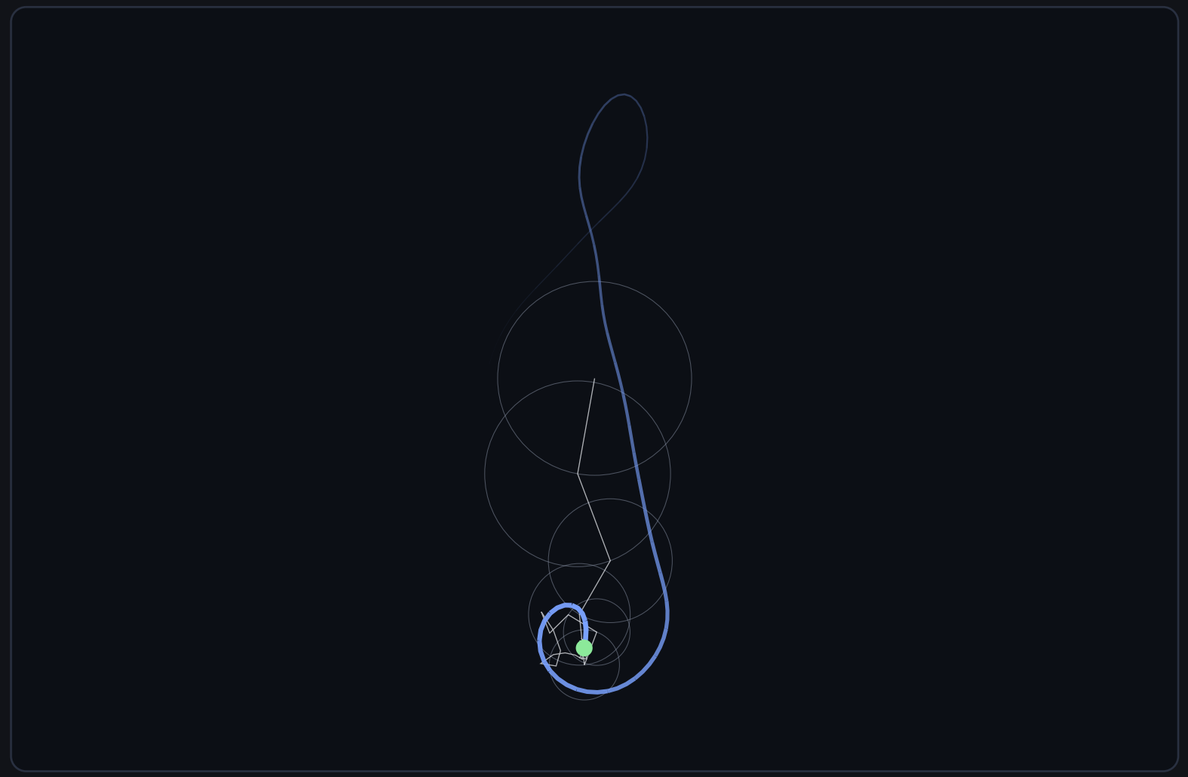
Three sources of keyframes: `keyframes` for the mark itself, `follow` for a path, and — next page — the mark's own states.

States played over time

Give an object an `anim:` with neither `keyframes` nor `follow`, and the **states of the drawings inside it become the keyframes**: stepping turns into playing.

```
#import "@preview/komet:0.2.0" as komet

#let coef = {                // the M largest terms of the DFT
  let n = clef.len()
  let out = ()
  for (k, z) in komet.fft(clef).enumerate() {
    let (re, im) = (z.at(0) / n, z.at(1) / n)
    out.push((n: if k * 2 <= n { k } else { k - n },
              re: re, im: im, r: calc.sqrt(re * re + im * im)))
  }
  out.sorted(key: e => -e.r).slice(0, M)
}
// epicycles(j) draws the chain at t = 2πj/S over the outline,
// whose pieces are lit by how far behind the pen they lie
// and inked by how much of them the pen has crossed
#let clef = cetz.canvas({
  let (states,) = cz
  rect((-0.83, -1.43), (0.83, 1.34))   // pin the box
  states(play: (duration: 6000), ..range(S + 1).map(epicycles))
})
#clef
```



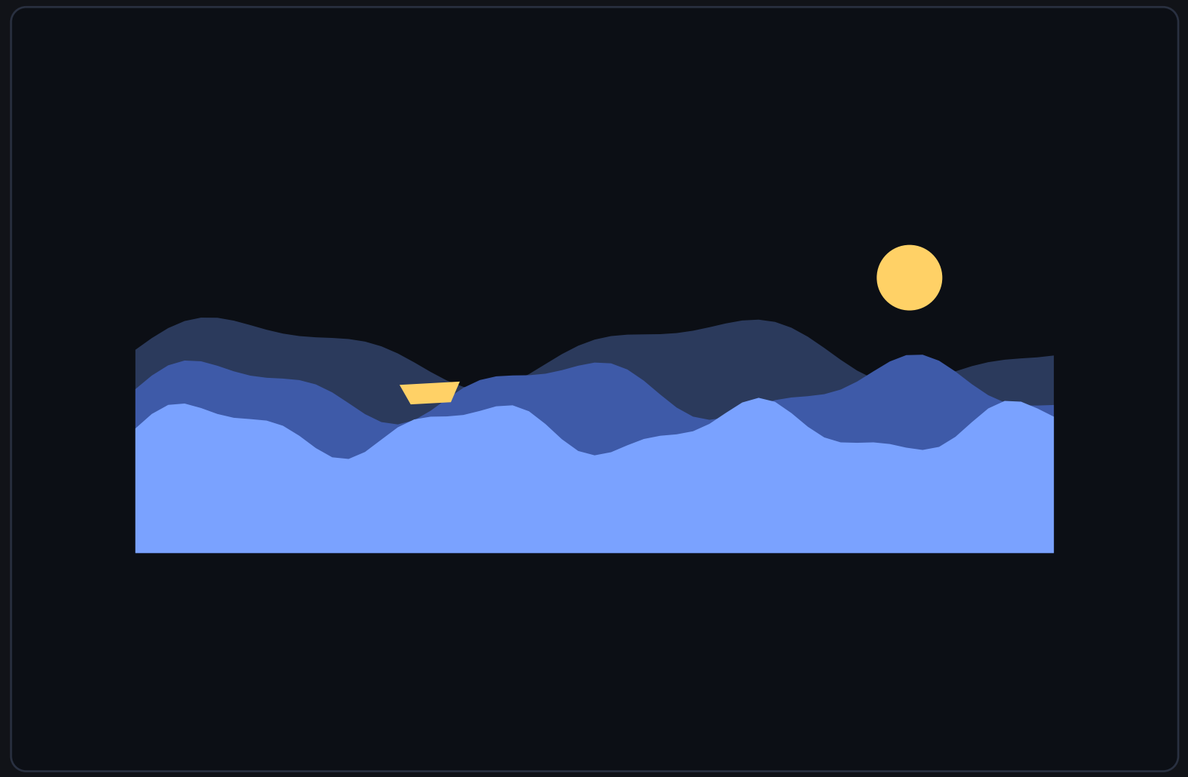
An object animated this way counts no steps. The states are a sampling rate, and between two of them every number moves at a constant rate: anything turning faster than S/2 is aliased, and geometry re-listed per state changes velocity at every keyframe. Hence 45 vectors, and a trail that is fixed pieces with moving light and moving ink.

A seamless loop

The same mechanism, twenty-five states, and one rule of composition: make the first state and the last state equal and the loop has nowhere to jump.

```
#let sea = cetz.canvas({
  let (states,) = cz
  rect(…)                                // box and sun stay put
  circle((11.8, 2.2), radius: .5, fill: gold)
  states(play: (duration: 4000), ..range(0, 25).map(k => {
    let phi = k / 24 * 2 * calc.pi     // 0 … 2π over 25 states
    let surf(a, w, v, y0) = x => y0
      + a * calc.sin(w * x + v * phi)
      + a * .35 * calc.sin(2.3 * w * x - 2 * v * phi)
    …three layers, then the boat on the front one…
  }))
})

#sea
```



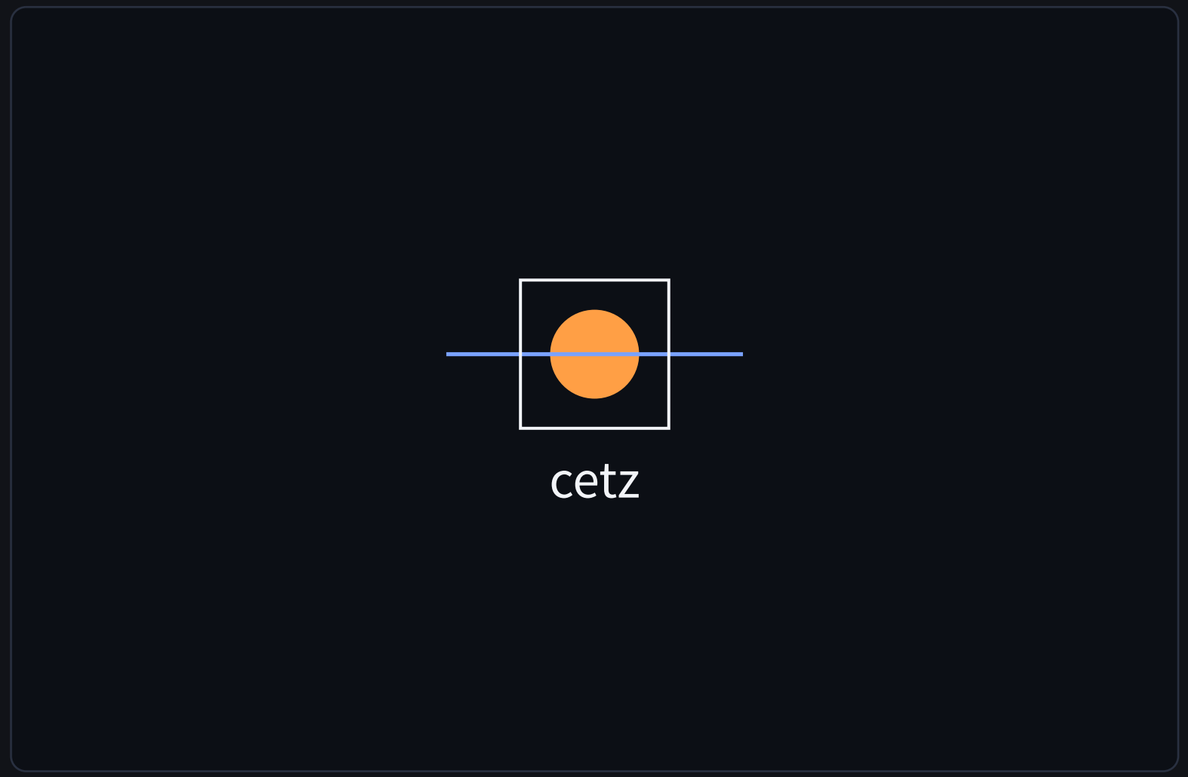
`iterations` is infinite by default; animations pause when the frame is left.

CeTZ

`cetz.canvas` exports native SVG paths, so a canvas in a `mark` pairs by key exactly as text does — and its nodes are what an element animation interpolates.

```
// CeTZ has a `mark` of its own (arrow heads), so
// `import cetz.draw: *` shadows this one — by name.
#import "@preview/cetz:0.5.2"

#let fig(r) = cetz.canvas({
  import cetz.draw: circle, rect, content
  circle((0, 0), radius: r)   // cetz's circle here;
  rect((-1, -1), (1, 1))      // std.circle for Typst's
  content((0, -1.7), [cetz])
})
#slide(title: "CeTZ", [#mark("fig")[#fig(0.6)]],
                      [#mark("fig")[#fig(1.2)]])
```



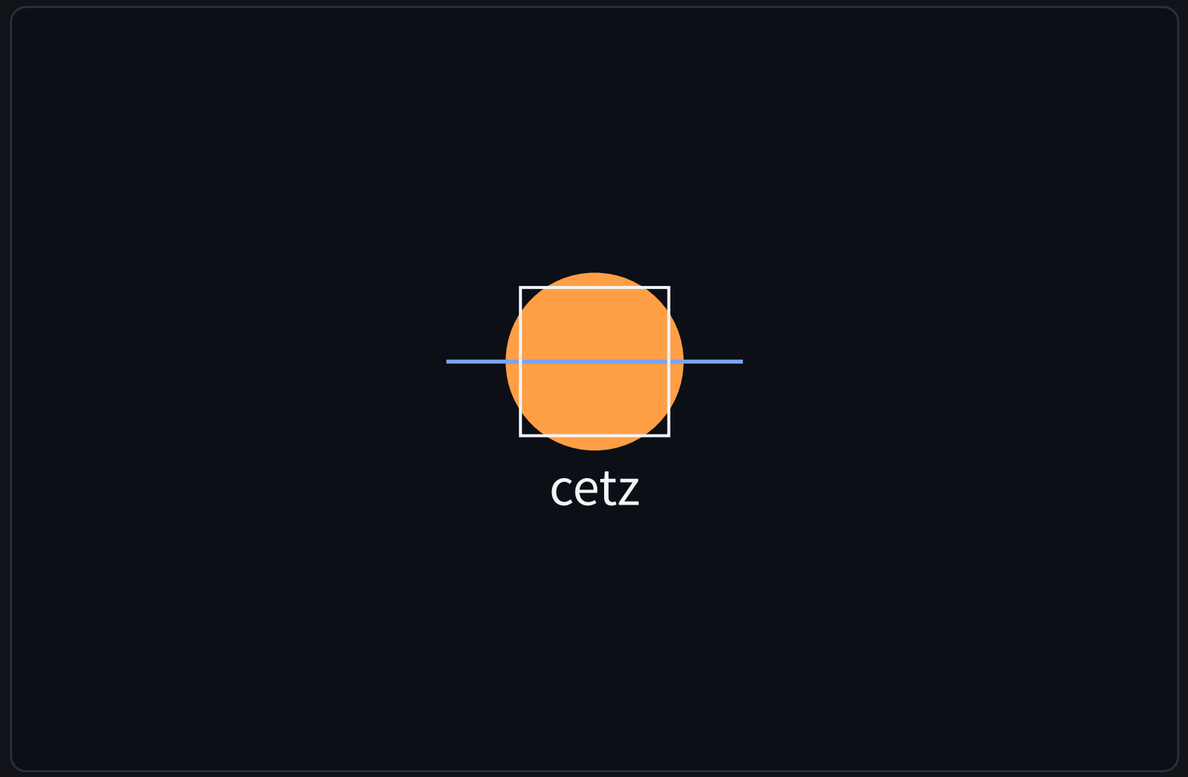
The bounding box follows what is drawn: pin it with an invisible `rect(…, stroke: none)`.

CeTZ

`cetz.canvas` exports native SVG paths, so a canvas in a `mark` pairs by key exactly as text does — and its nodes are what an element animation interpolates.

```
// CeTZ has a `mark` of its own (arrow heads), so
// `import cetz.draw: *` shadows this one — by name.
#import "@preview/cetz:0.5.2"

#let fig(r) = cetz.canvas({
  import cetz.draw: circle, rect, content
  circle((0, 0), radius: r)   // cetz's circle here;
  rect((-1, -1), (1, 1))      // std.circle for Typst's
  content((0, -1.7), [cetz])
})
#slide(title: "CeTZ", [#mark("fig")[#fig(0.6)]],
                      [#mark("fig")[#fig(1.2)]])
```



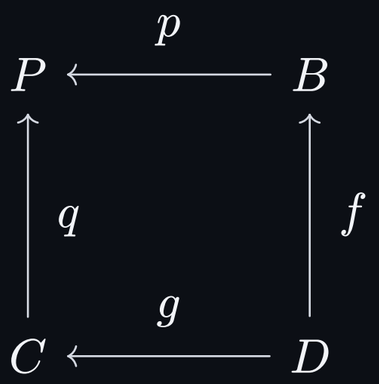
The bounding box follows what is drawn: pin it with an invisible `rect(…, stroke: none)`.

Third party packages

A slide is an `html.frame`, and inside a frame Typst's target is the **paged** one — so theorion and fletcher take their PDF branch and hand us a picture, which is what a mark can move.

```
#import "@preview/theorion:0.6.0": *
#import cosmos.simple: *
#import "@preview/fletcher:0.5.8" as fletcher: diagram, node, edge

#let duo(rev) = diagram({
  let arrow(a, b, l) = if rev { edge(b, a, l, "->") }
                       else   { edge(a, b, l, "->") }
  node((0, 0), $P$); node((1, 0), $B$)
  node((0, 1), $C$); node((1, 1), $D$)
  arrow((0, 0), (1, 0), $p$); …
})
#slide[#tween(duo(false), duo(true))]
```



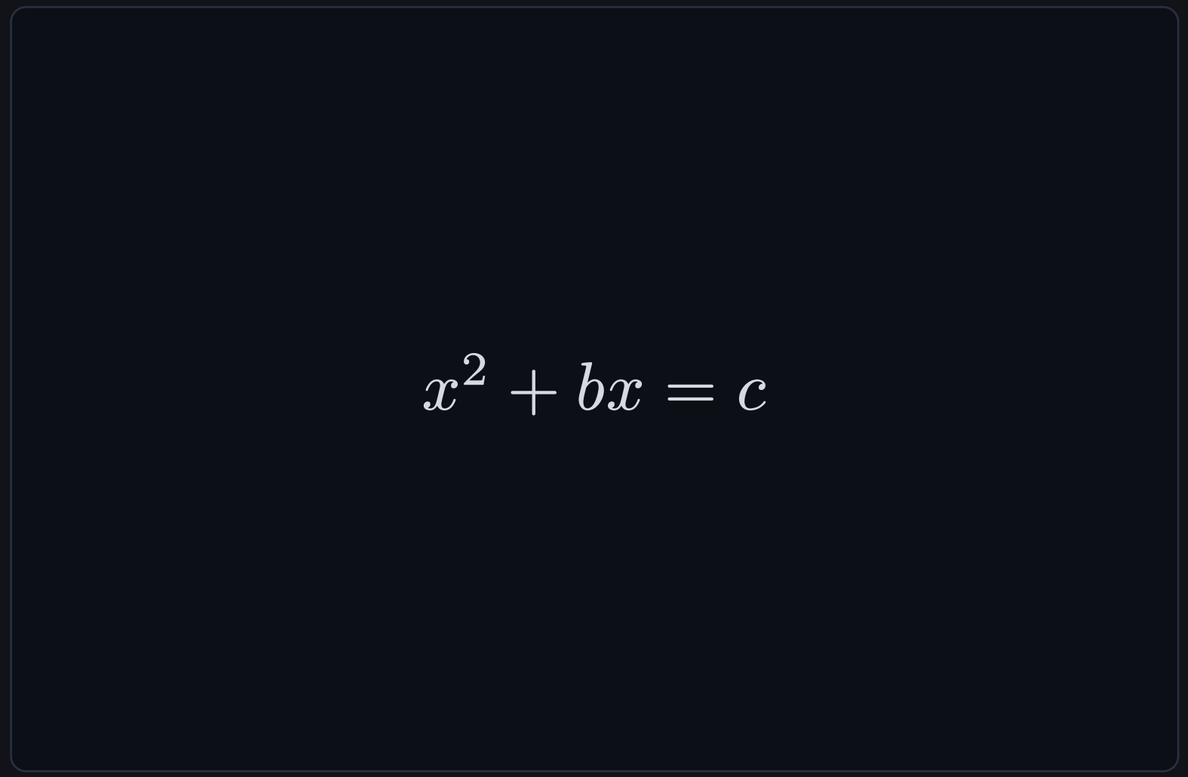
Mark what the package takes as **content**; its arrows are drawn, so keep the geometry still.

Inside a formula

A mark is content and a formula is made of content, so a term is marked where it stands — handed over **as an equation**, not as bare math.

```
#let sq = mark.with("sq")
#let rhs = mark.with("rhs")
#let half = mark.with("half", transition: "rise")
#let add = mark.with("add", transition: "rise")

#slide(title: "Completing the square",
  $ #sq($x^2$) + #sq($b x$) = #rhs($c$) $,
  $ #sq($x^2$) + #sq($b x$) + #half($(b/2)^2$)
      = #rhs($c$) + #add($(b/2)^2$) $,
  $ #sq($(x + b/2)^2$) = #rhs($c$) + #add($(b/2)^2$) $,
)
```



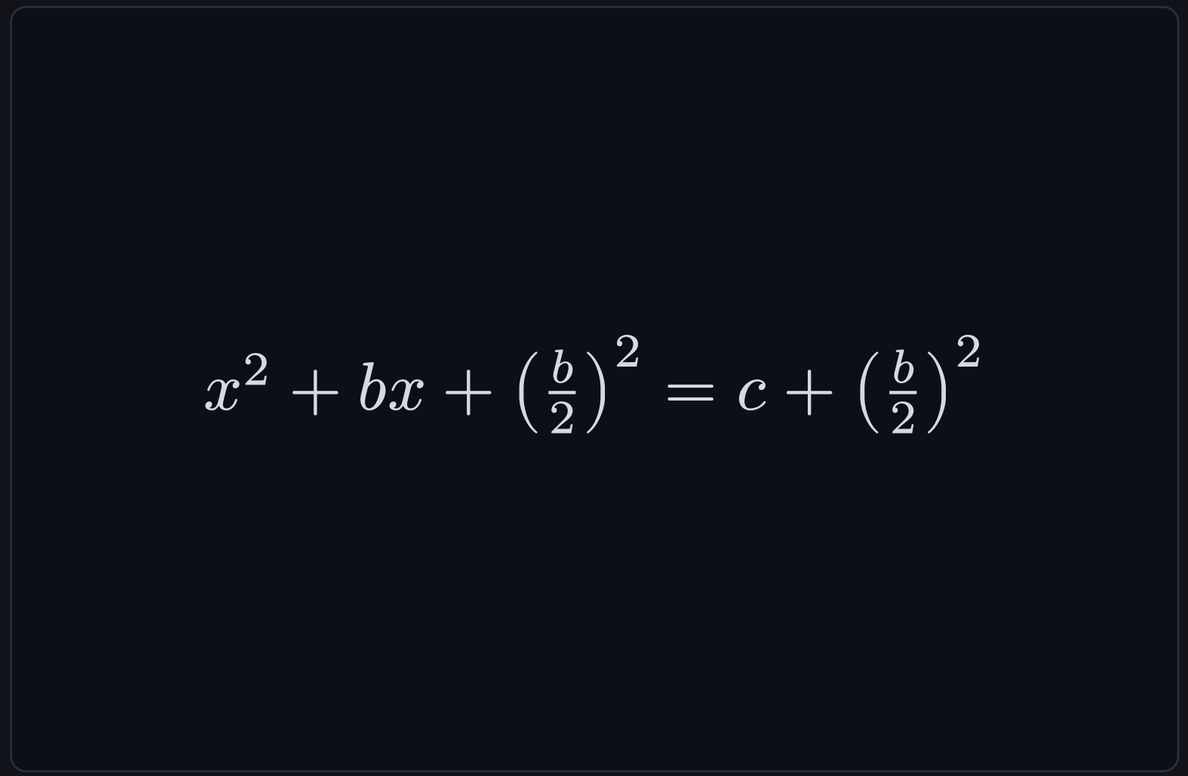
A box lays content out as markup, which would set `b x` upright; as an equation it is untouched.

Inside a formula

A mark is content and a formula is made of content, so a term is marked where it stands — handed over **as an equation**, not as bare math.

```
#let sq = mark.with("sq")
#let rhs = mark.with("rhs")
#let half = mark.with("half", transition: "rise")
#let add = mark.with("add", transition: "rise")

#slide(title: "Completing the square",
  $ #sq($x^2$) + #sq($b x$) = #rhs($c$) $,
  $ #sq($x^2$) + #sq($b x$) + #half($(b/2)^2$)
      = #rhs($c$) + #add($(b/2)^2$) $,
  $ #sq($(x + b/2)^2$) = #rhs($c$) + #add($(b/2)^2$) $,
)
```



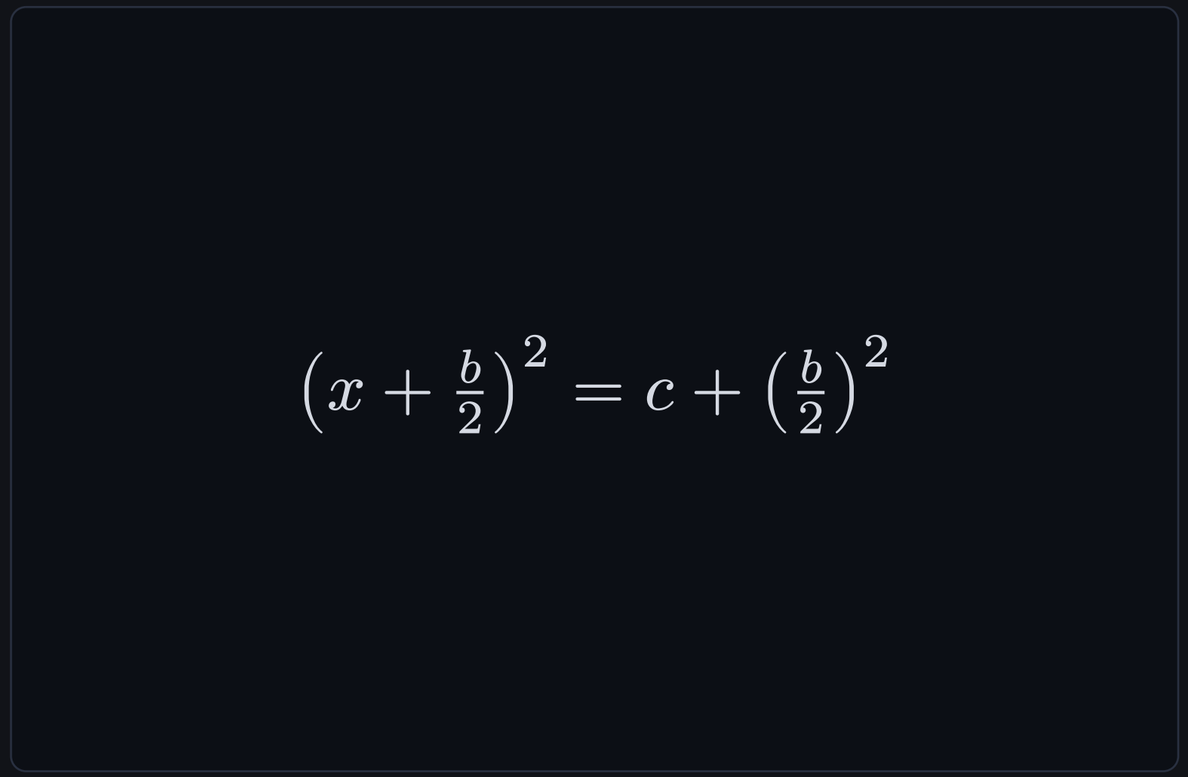
A box lays content out as markup, which would set `b x` upright; as an equation it is untouched.

Inside a formula

A mark is content and a formula is made of content, so a term is marked where it stands — handed over **as an equation**, not as bare math.

```
#let sq = mark.with("sq")
#let rhs = mark.with("rhs")
#let half = mark.with("half", transition: "rise")
#let add = mark.with("add", transition: "rise")

#slide(title: "Completing the square",
  $ #sq($x^2$) + #sq($b x$) = #rhs($c$) $,
  $ #sq($x^2$) + #sq($b x$) + #half($(b/2)^2$)
      = #rhs($c$) + #add($(b/2)^2$) $,
  $ #sq($(x + b/2)^2$) = #rhs($c$) + #add($(b/2)^2$) $,
)
```



A box lays content out as markup, which would set `b x` upright; as an equation it is untouched.

The deck

`deck` is the document. Its arguments are the only global settings there are, and the PDF and the HTML both read them.

```
#deck(
  title: "My talk",
  width: 1280pt, height: 720pt,
  pdf: auto,                  // the .pdf beside the .html
  duration: 700,              // milliseconds
  easing: (0.32, 0.72, 0, 1),
  transition: "fade",
  theme: auto,                // player chrome only
  fill: rgb("#111318"),
  font: ("DejaVu Sans", "Noto Sans CJK SC"),
  body,
)

#slide(title: "…", note: [speaker notes], …frames)
```

`title`	document title
`width / height`	layout size; the PDF page and the player's aspect ratio
`pdf`	the toolbar's download link; none removes the button
`duration`	default transition duration, in milliseconds
`easing`	the four numbers of a cubic Bézier
`transition`	default page-to-page effect
`theme`	light or dark player chrome
`fill`	layout background, painted the same in both formats
`font`	font stack, glyph-by-glyph fallback
`slide(title:)`	thumbnail caption; never in the layout
`slide(note:)`	speaker notes; HTML only

`title` and `note` are content, not just strings: paragraphs, lists and emphasis all render in the notes.

Presenting

The player is part of the deck: no server, no build step. Open the HTML file and it is a presentation.

```
window.vit
// { go, next, prev, index, total,
//   step, steps, speed, mode, deck }
// step, speed and mode are writable;
// mode is "desk" | "present" | "overview"

const deck = document.querySelector('.vit-deck')
// the new position is in the DOM, and then
// it has finished moving — one for one,
// e.detail is { index, step } for both
deck.addEventListener('vit:move-ready', on)
deck.addEventListener('vit:move-done', on)
```

→ ↓ Space	next position: next step, next frame, next page
← ↑ ⌫	previous position
Home End	first / last page
1–9	jump to a page
Esc / Enter	desk ⇄ presenting
o / a	overview; a dot per position, hover to peek
l	laser pointer
b / .	black screen
− = 0	slower / faster / reset, kept in localStorage
s	speaker view: next page, notes, timer
f	full screen
?	this table

The desk is what opens; its preview is a copy of this HTML at a hash, so it plays the real thing.

Choosing

Nine calls, and the question each of them answers. When two would do, the one that keeps the layout still is the one that morphs.

```
mark("key")[…]                  // this thing is that thing
tween(s0, s1, …)                // one drawing, N parameters
mark(transition: …)             // how this thing enters
slide(a, b, c)                  // frames, written out
build(a, b, c)                  // frames that accumulate
reveal(n, (step, at) => …)      // frames that stay put
layers(key, meet: …, a, b)      // an assembly that grows
mark(key, anim: …)              // it moves on its own
deck(transition:, duration:, …)// how pages turn
```

Something is in two places?	`mark`, one key
A page grows a line at a time?	`build` — the later parts push the layout
Parts arrive but nothing may shift?	`reveal` — what is to come holds its space
A drawing gains something outside itself?	`layers` — what was there glides as one picture
A curve must bend, a pose must change?	`tween` — a transition only cross-fades
It must move with nobody pressing?	`tween(play:)`, `waapi.animate`

Out of reach: z-order, silently clipped overflow, a bitmap halfway through a big size change.